Codex Handoff Notes

Updated: 2026-10-22

Project summary

DcRB is a Discord bot that exposes local computer control through slash commands and Windows/Linux message commands beginning with `--`. Current project version is declared as `0.0.1` in `dcrbs_version.py`.

Recent work and accepted decisions

- Unified startup and message `--command(...)` parsing in `dcrbs_command_dispatch.py`; duplicate command names now raise a collision error instead of silently replacing one another.
- Kept `dcrbs.pyw` as a small wrapper around `dcrbs.main()`.
- Consolidated Linux desktop and TTY module configuration into `dcrbs_modules_linux.json`.
- Added guild-scoped access control. The local highest-administrator password is prompted for on first `dcrbs` startup, stored as a salted scrypt hash in `dcrbs_owner_password.hash`, and entered into Discord through the private `/admin_setup` modal.
- The Discord owner session lasts 10 minutes. `/access_scope` adds, removes, and lists per-guild user and role assignments; `/access_logout` ends the caller's session.
- Current user decision: remove every command-specific/secondary password check. All functional commands are authorized through the administrator-assigned access levels. Only the local owner password remains for `/admin_setup`.
- Current user-selected default tiers: level 1 for high-impact/system startup operations and shell commands (`cmd`, including Linux Bash, and `powershell`); level 2 for keyboard command execution; level 3 for regular read/control operations. `startup`, `shell_startup`, `reg_startup`, `reg_startup_remove`, and `audit_config` are fixed to level 1 even when older config files assign a lower level. Unclassified commands default to level 1.
- Current user decision: `/stop(duration)` pauses new ordinary commands, cancels managed looping and keyboard tasks, and pauses blacklist enforcement; it leaves shell processes running and resumes command access plus blacklist enforcement when the timer expires. `/panic_stop` performs the same pause and terminates tracked shell process trees until a level-1 `/resume` is used. The hard lock persists in `dcrbs_panic_stop.lock` across restarts, and saved startup commands are skipped while locked. Stopped loops do not restart automatically.
- Updated application-command error handling in `dcrbs.py` for Discord error 10062 (`Unknown interaction`): expired interactions are logged as warnings without attempting a second, doomed response. For other command errors, the handler uses a followup when the interaction was already acknowledged and catches HTTP failures while reporting.
- `1.log` showed `/admin_setup`'s modal response arriving after Discord had expired the interaction, followed by a second 10062 from the global error handler. An expired interaction cannot be used to open the modal or send a reply; retry `/admin_setup` after the bot reconnects.

Startup and module graph

`dcrbs.py` initializes the owner hash, reads `dcrbs_token.txt`, then launches `BotRunner`. `setup_hook()` selects `dcrbs_modules_linux.json` on Linux and `dcrbs_modules.json` otherwise, imports each listed module and calls its `setup(bot)`. It then adds `dcrbs_auth.generalCheck` and `dcrbs_access_control` checks before syncing slash commands. Startup stops if `dcrbs_access_control` did not load.

Windows configuration loads `dcrbs_auth`, `dcrbs_control`, `dcrbs_system`, `dcrbs_events`, and `dcrbs_access_control`. Linux configuration loads `dcrbs_auth`, `dcrbs_linux`, `dcrbs_events`, and `dcrbs_access_control`. The non-Linux branch currently uses Windows modules, so macOS deployment is not recommended.

`dcrbs_events` passes `--` messages to `dcrbs_command_dispatch.dispatchTextCommands`; `dcrbs.py` uses the same dispatcher for saved startup text. Dispatcher discovers callbacks from loaded Cogs, converts simple typed arguments, and applies command checks for normal text messages. Saved startup text runs with `runChecks=False` after it was configured through `/startup`. Slash and recognized `--` commands queue metadata-only Google Forms audit records; shutdown closes tracked shell process trees and flushes pending audit submissions.

Permission model

- Level 1: explicitly assigned user IDs only.
- Level 2: user IDs or role IDs.
- Level 3: user IDs or role IDs.
- The lowest numeric level grants the greatest access. A user assigned more than one level receives the strongest one.
- In DMs, only a user assigned level 1 in at least one configured guild is authorized; level 2/3 access remains guild-only. Guild-scoped management commands and `/startup` remain guild-only.
- Permission configuration is per guild in `dcrbs_access_control.json`; command levels are configurable in its `command_levels` map.
- `dcrbs_audit_config.json` holds the deployment's Google Forms response URL and one long-answer field ID. `/audit_config` is available to level 1 administrators and writes this file. Audit records contain UTC time, command name, user ID, guild ID, and source, never arguments or output.
- `/admin_setup`, `/access_scope`, and `/access_logout` are management commands and are excluded from ordinary command checks. Scope changes require the caller's valid owner session.
- Do not reintroduce password parameters or per-command password checks. The owner hash is the only password credential used by current code.

Default command levels in `dcrbs_access_control.py`:

- 1: `stop`, `panic_stop`, `resume`, `reject`, `token`, `shutdown`, `startup`, `shell_startup`, `reg_startup`, `reg_startup_remove`, `powershell`, `cmd`, `audit_config`
- 2: `key`, `keyboard`, `key_wait`

- 3: `web`, `blackade`, `unblackade`, `txt`, `msg`, `close`, `close_loop`, `close_stop`, `monitor_off`, `monitor_on`, `screenshot`, `screenshot_notify`, `device`, `volume`, `task_list`, `cmdlist`, `help`

Command behavior by module

- `dcrbs_auth.py`: `/stop(duration)`, `/panic_stop`, `/resume`, `/reject`, `/token(token)`, `/audit_config(form_url, entry_id)`, `/help`. `/stop` pauses ordinary commands and managed loops; `/panic_stop` also terminates tracked shell processes and locks command access until `/resume`. Access-management commands remain available. `/reject` exits the process.
- `dcrbs_control.py` (Windows): text-file display, message box, web launch, keyboard input, window closing, monitor control, screenshots, and process blacklist handling.
- `dcrbs_system.py` (Windows): device information, CMD and PowerShell, volume, shutdown, startup config, Startup-folder shortcut, registry startup and removal, process/window list and command list.
- `dcrbs_linux.py`: Linux device information, Bash command, shutdown, and process list.
- `dcrbs_events.py`: Windows/Linux text-command entry point.

Local files and dependencies

- Required input: `dcrbs_token.txt`.
- Owner credential: `dcrbs_owner_password.hash`. First setup requires an interactive terminal and a 12 – 128 character password. A legacy `owner_password.txt` is migrated to the hash and removed at startup.
- Runtime state: `dcrbs_access_control.json` (permissions) and `dcrbs_startup.json` (saved startup command and guild ID).
- Google Forms audit destination: `dcrbs_audit_config.json`; the deployer supplies a public Google Forms response endpoint and field entry ID.
- `dcrbs_panic_stop.lock` persists the emergency control lock until `/resume` clears it.
- `auth_secret.txt` and `password.json` are legacy files. Current Python code no longer reads, creates, or writes them. Their `.gitignore` entries are intentionally retained to protect leftover local copies.
- Dependencies are listed in `requirements.txt`: `discord.py`, `aiohttp`, `psutil`, `pyautogui`, `mss`, `pygetwindow`, `pycaw`, `keyboard`, `Pillow`.
- `dcrbs_usage.csv` is the live `/cmdlist` content. Keep it aligned with command signatures; do not advertise removed commands such as `--kill` or obsolete password arguments.
- Existing ZIP files under `packages/` are historical archives. `packages/dcrbs_0.0.1.zip` is the pip-installable source package and is not imported by the active entry point.

Current task status

Current user confirmation: `--` text commands respond in DMs but not in server channels in the current deployment. This is acceptable; README now directs users to use text commands in DMs and slash commands in guild channels. The code does not enforce DM-only handling. If guild text-command support is requested later, check the Discord Developer Portal Message Content Intent and channel permissions.

Completed in the current worktree:

1. Removed secondary password checks, password parameters, old password generators/storage helpers, and `/change_password`.
2. Updated `dcrbs_usage.csv` for the changed `/stop`, `/reject`, and `/token` signatures and removed its stale `--kill` entry.
3. Rewrote `README.md` with setup, module dependencies, platform behavior, permission levels, commands, and runtime files.
4. Created this handoff file.
5. Made the global application-command error handler tolerate expired Discord interactions and avoid the secondary unhandled task exception shown in `1.log`.
6. Fixed shell and startup configuration commands at level 1; persisted migration raises older configured levels.
7. Added Google Forms metadata auditing for slash and text commands, `/audit_config`, and the ignored `dcrbs_audit_config.json` deployment file.
8. Clean up tracked shell process trees during orderly Bot shutdown, including Ctrl+C, and document that macOS deployment is not recommended.

No tests have been run. After edits, inspect references for old password logic and run `git diff --check`; do not launch the bot during static handoff review because it may prompt for a local owner password or connect to Discord.